Übungsblatt 7: Performance und Pipelining

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Latenz und Durchsatz neu berechnen (12 Punkte)

Fünf Stufen IF, ID, EX, MEM, WB. Angenommene reine Stufenzeiten: 180, 140, 220, 160, 100 ps; effektiver Registeroverhead 30 ps je Taktpfad. Single Cycle durchläuft alle Bereiche in einem Takt, die Pipeline trennt alle fünf Stufen. Keine Stalls, anfangs leer. Gezählt wird vom ersten IF bis zum letzten WB.

1. Berechnen Sie beide Taktperioden und begründen Sie Summe beziehungsweise Maximum. (3 P)
2. Bestimmen Sie die Einzellatenzen. Warum entscheidet deren Vergleich noch nicht über den Durchsatz? (2 P)
3. Zwölf unabhängige Instruktionen: Takte, endlicher CPI, Gesamtzeiten und Speedup? (5 P)
4. Bestimmen Sie den Speedup-Grenzwert für sehr große N. Welche zwei Bedingungen fehlen für Faktor fünf? (2 P)

Musterlösung

1. $T_{\text{single}} = 180+140+220+160+100+30 = 830$ ps; $T_{\text{pipe}} = \max(180, 140, 220, 160, 100)+30 = 250$ ps. Zeiten je 1 P, Begründung 1 P.
2. Latenzen 830 ps und $5 \cdot 250 = 1250$ ps (1 P). Überlappung kann den Durchsatz trotz längerer Einzellatenz erhöhen (1 P).
3. Single: C = 12, CPI = 1, Zeit 9960 ps. Pipeline: C = 16, CPI = 16/12, Zeit 4000 ps. Speedup = 2,49. Takte 1 P, CPI 1 P, Zeiten je 1 P, Speedup 1 P.
4. Grenzwert $830/250 = 3{,}32$ (1 P). Ungleich lange Stufen und relevanter Registeroverhead verhindern hier Faktor fünf (zusammen 1 P).

Aufgabe 2: Vier abhängige Instruktionen (12 Punkte)

Anfangs leere Fünfstufen-Pipeline, Forwarding aus EX/MEM und MEM/WB; pro Takt höchstens ein neuer Befehl. Ein Load braucht eine MEM-Stufe, seine Daten sind erst danach bereit. Benötigte nicht genannte Registerwerte sind unbekannt. t1 = 15; s0 zeigt auf zwei gültige Wörter, das zweite hat Wert −9.

```
lw  t2, 4(s0)        # I1
sub t3, x0, t2       # I2
and t4, t3, t1       # I3
add a0, t4, t3       # I4
```

1. Zeichnen Sie das vollständige Stufenraster. Markieren Sie gehaltene IF-/ID-Stufen und die Bubble. (5 P)
2. Geben Sie die Quelle samt Wert jedes abhängigen EX-Operanden an; beachten Sie die zwei unterschiedlichen Produzenten von I4. (3 P)
3. Bestimmen Sie t2, t3, t4 und a0 nach Abschluss. (2 P)
4. Nennen Sie N, Gesamttakte und endlichen CPI. Zählt die Bubble zu N? (2 P)

Musterlösung

Eintrag / Takt	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	ID*	EX	MEM	WB		
I3			IF	IF*	ID	EX	MEM	WB	
I4					IF	ID	EX	MEM	WB
Bubble				EX	MEM	WB			

1. Stern = gehalten. Je richtiger Instruktionszeile 1 P, Bubble 1 P.
2. I2 EX5: −9 aus MEM/WB. I3 EX6: 9 aus EX/MEM. I4 EX7: t4 = 9 aus EX/MEM und t3 = 9 aus MEM/WB. Die Load-Adresse ist kein Ersatz für die geladenen Daten. I2 und I3 zusammen 1 P, beide I4-Quellen je 1 P.
3. t2 = −9, t3 = 9, t4 = 9, a0 = 18. Je 0,5 P.
4. N = 4, C = 9, CPI = 2,25; Bubble zählt nicht. Je Angabe 0,5 P.

Aufgabe 3: Der richtige Produzent (8 Punkte)

Forwarding wie in Aufgabe 2; Anfangswerte stehen vor dem Kern bereits bereit.

1. t1 = `0x12`, t2 = 5, t4 = 15. Wenn die letzte Zeile EX erreicht, welche Quelle liefert t0 und welcher t3-Wert entsteht? (3 P)

```
or   t0, t1, t2
addi t0, t0, -4
and  t3, t0, t4
```

2. Unabhängig davon t1 = 18, t4 = 15: `addi x0, t1, 9` gefolgt von `or a0, x0, t4`. Warum darf 27 nicht weitergeleitet werden? Welches a0 entsteht? (3 P)
3. Ein anderer Schreiber steht in WB, sein Leser nach zwei unabhängigen Instruktionen gleichzeitig in ID. Im Papiermodell wird ein am Taktende geschriebener Wert erst im Folgetakt lesbar. Genügen ausschließlich die EX-Forwarding-Pfade? Nennen Sie Behandlung und eine alternative Registerbankkonvention. (2 P)

Musterlösung

1. OR liefert 23, ADDI danach 19. Der jüngere Produzent in EX/MEM muss gegenüber dem älteren in MEM/WB gewinnen. `t3=19 AND 15=3`. Wertefolge 1 P, Quelle und Vorrang 1 P, Ergebnis 1 P.
2. x0 verwirft den Rechenwert 27. Ein Leser von x0 braucht null; rd = x0 muss daher vom Forwarding ausgeschlossen werden. a0 wird 15. Erklärung 2 P, Ergebnis 1 P.
3. Nein: Im nächsten EX-Takt ist der alte Schreiber bereits aus MEM/WB verschwunden. Ein Takt Halt in ID mit erneutem Lesen oder ein zusätzlicher WB-ID-Bypass löst den Fall (1 P). Eine Registerbank mit Schreiben vor Lesen im selben Takt wäre eine andere Konvention (1 P).

Aufgabe 4: Zwei falsche Zustandsänderungen verhindern (10 Punkte)

Branch-Auflösung in EX, vorher sequenzieller Fetch; bei genommenem Branch werden die beiden jüngeren Einträge verworfen. Vergleichswerte sind bereit. Anfang: a0 = a1 = −2, t2 = `0x55`, a2 = 0. s0 = B zeigt auf ein beschreibbares Wort `0x33`; auch B + 4 ist gültig.

```
beq  a0, a1, Ziel
addi s0, s0, 4
sw   t2, 0(s0)
Ziel:
xori a2, t2, 15
```

1. Zeichnen Sie das Raster bis zum WB der Zielinstruktion und markieren Sie den Flush. (4 P)
2. Geben Sie s0, Speicherwort bei B und a2 danach an. Warum genügt es nicht, nur den Fetch umzulenken? (2 P)
3. Ein anderer Lauf schließt 240 Instruktionen ab, hat 17 zusätzliche Wartezyklen und 40 Branches, davon 15 genommen. Zusatzkosten überschneiden sich nicht. Bestimmen Sie C, CPI und Zeit bei 250 ps; Füllen und Leeren sind einzubeziehen. (4 P)

Musterlösung

Eintrag / Takt	1	2	3	4	5	6	7	8
beq	IF	ID	EX	MEM	WB			
addi s0		IF	ID	verworfen				
sw			IF	verworfen				
xori a2				IF	ID	EX	MEM	WB

1. Je Zeile 1 P. Ab Grenze zu Takt 4 sind beide jüngeren Einträge ungültig.
2. s0 bleibt B, Wort bei B bleibt `0x33`, a2 wird `0x5A` = 90 (1 P). Bereits geholte falsche Arbeit darf später weder Pointer noch Speicher verändern (1 P).
3. $C = 240 + 4 + 17 + 2 \cdot 15 = 291$; CPI = 1,2125; Zeit = 72750 ps = 72,75 ns. Takte 2 P, CPI 1 P, Zeit 1 P. Branches gehören bereits zu den 240 Instruktionen.

Aufgabe 5: Vier Befehle sinnvoll umordnen (10 Punkte)

Das Wort bei 4(s0) ist −9; t1 = 30, a5 = 1, a6 beliebig. Forwarding und Load-Use-Erkennung wie zuvor, Kernzählung vom ersten IF bis zum letzten WB.

```
lw   t2, 4(s0)
sub  a0, t1, t2
addi a5, a5, 2
xor  a6, a6, a6
```

1. Ordnen Sie genau diese vier Instruktionen so um, dass der Load-Use-Stall entfällt. Begründen Sie die Unabhängigkeit der verschobenen Arbeit. (3 P)
2. Bestimmen Sie Endwerte t2, a0, a5, a6 sowie Takte und Speedup vor/nach der Umordnung. (3 P)
3. Warum ist ein zusätzlich eingefügtes `addi a5, a5, 2` kein gleichwertiges Scheduling? (2 P)
4. Würde stattdessen eine Pointererhöhung von s0 als unabhängige Füllinstruktion gelten? Unterscheiden Sie Verschieben vor den Load und zwischen Load und Nutzung. (2 P)

Musterlösung

1. Beispielsweise lw, addi a5, sub, xor a6. Alternativ kann xor a6 die Lücke füllen. Die verschobene Arbeit liest weder Loadwert noch Basis und verändert diese nicht. Reihenfolge 1 P, Register-/Speicherbegründung 2 P.
2. t2 = −9, a0 = 39, a5 = 3, a6 = 0 (zusammen 1 P). Vorher 9, danach 8 Takte (1 P). Speedup $9/8 = 1{,}125$ (1 P).
3. Es wären fünf statt vier Instruktionen und a5 würde 5 statt 3. Befehlszahl und geändertes Ergebnis je 1 P.
4. Vor dem Load verschiebt die Erhöhung die gelesene Adresse und ist hier nicht zulässig (1 P). Zwischen Load und sub darf sie bei ansonsten gleichen Voraussetzungen stehen, weil sub nur das bereits geladene Wort benötigt (1 P).

Aufgabe 6: Eine wechselnde Branch-Folge (8 Punkte)

Isolierter Branch mit Folge N, T, N, N, T, T. T = genommen, N = nicht genommen. Zieladresse ist rechtzeitig verfügbar, kein geteilter Richtungseintrag; je Vorhersagefehler zwei Zusatztakte.

1. Einbitprädiktor startet mit T und speichert nach jedem Branch die tatsächliche Richtung. Bestimmen Sie Vorhersagen, Fehlerpositionen, Endzustand und Zusatzzyklen. (4 P)
2. Zweibitzähler startet in 10. 00/01 sagen N, 10/11 sagen T; T erhöht sättigend, N verringert sättigend. Bestimmen Sie Vorhersagen, Zustandsfolge und Fehlerzahl. (3 P)
3. Was zeigt der Vergleich über eine pauschale Überlegenheit des Zweibitmodells? (1 P)

Musterlösung

1. Vorhersagen T, N, T, N, N, T. Fehler an Position 1, 2, 3, 5; Endzustand T, acht Zusatzzyklen. Je gefragter Angabengruppe 1 P.
2. Vorhersagen T, N, T, N, N, N. Zustände nachher: 01, 10, 01, 00, 01, 10. Fünf Fehler, also zehn Zusatzzyklen. Vorhersagen 1 P, Zustände 1 P, Fehlerzahl 1 P.
3. In diesem Verlauf ist das Zweibitmodell schlechter. Verhalten hängt von Folge und Anfangszustand ab; eine allgemeine Überlegenheit folgt nicht. 1 P.

Aufgabe 7: Eigenständige Scheduling-Versuche in Ripes

1. Öffnen Sie Variante A und Variante B nacheinander in derselben RV32-Fünfstufen-Variante mit Forwarding und Hazard-Erkennung, C aus. Notieren Sie Version und Modell.
2. Sagen Sie die vier Endwerte am Prüfpunkt voraus. Starten Sie jede Datei neu.
3. Verfolgen Sie jeweils nur den Bereich Kern bis Ende_Kern. Die vorbereitenden NOPs und die Inspektionsschleife zählen nicht zu diesen vier Instruktionen.
4. Protokollieren Sie ersten IF, letzten WB, gehaltene Stufen sowie Wert und Herkunft der Subtraktionsoperanden.
5. Begründen Sie, ob eine Zeitdifferenz auf entfernten Stalls oder auf veränderter Arbeit beruht. Verwenden Sie den globalen Zähler nicht als ungeprüfte Kernzählung.

Musterlösung

Beide Dateien rechnen dieselbe Aufgabe aus Aufgabe 5: t2 = −9, a0 = 39, a5 = 3, a6 = 0. A hat im Papiermodell neun, B acht Kerntakte. Die Differenz ist genau der vermiedene Load-Use-Stall. Die vier vorbereitenden NOPs trennen die Initialisierung vom Kern; sie sind keine vom Hazard-Baustein erzeugten Bubbles. Bei abweichender Ripes-Registerbank- oder Stufenkonvention ist die tatsächliche Stufenhistorie maßgeblich.